

EXPERIMENT – 3

SURVEY REPORT ON EMERGING TRENDS IN LOW-CODE APPLICATION DEVELOPMENT

Name: Anu Shalini B

Register No.:240701043

1. AIM

To study the emerging trends in low-code application development, compare low-code development with traditional application development, identify its benefits and challenges, and compare commonly used low-code platforms.

2. TRADITIONAL APPLICATION DEVELOPMENT VS EMERGING TRENDS IN LOW-CODE APPLICATION DEVELOPMENT

Traditional application development usually requires developers to write source code for the user interface, business logic, database operations, APIs, testing, and deployment. It provides detailed control but normally requires more development time and programming knowledge.

Low-code application development uses visual interfaces, drag-and-drop components, reusable modules, workflows, templates, and automation. It reduces the amount of manual coding required.

Professional developers can also add custom code when advanced functionality is needed.

Key Emerging Trends

- AI-assisted development: AI can help generate application logic, suggest workflows, create formulas, and assist developers during application building.
- Visual development: Drag-and-drop components and visual designers allow applications to be created with less manual coding.
- Workflow automation: Business processes can be automated using visual workflows, triggers, approvals, and notifications.
- Reusable components: Templates, modules, connectors, and pre-built components reduce repeated development work.

- Cloud-based development: Applications can be developed, deployed, and maintained through cloud platforms.
- API and system integration: Low-code platforms increasingly provide connectors and APIs to communicate with databases and external services.
- Mobile-first development: Many platforms support responsive or mobile applications from the same development environment.
- Citizen development: Non-professional developers can create simple business applications with guidance and governance.

Traditional Development vs Low-Code Development

Aspect Traditional Development Low-Code Development

Coding Requires substantial manual

Aspect	Traditional Coding	Low-Code / No-Code
Coding	Uses more coding	Uses visual tools with less coding
Development Speed	Usually slower for standard applications	Usually faster for standard applications
Programming Skill	Higher programming knowledge is generally required	Basic to moderate programming knowledge may be enough for many tasks
Customization	Very high control and advanced customization	Good customization; advanced customization cases may require custom code
Maintenance	Developers manage much of the codebase	Platform features can simplify maintenance
Integration	APIs and integrations are commonly coded/configured manually	Pre-built connectors and APIs can simplify integration
Target Users	Mainly professional developers	Professional developers and citizen developers

3. BENEFITS AND CHALLENGES IN LOW-CODE

Benefits

- Faster application development because visual components and reusable templates reduce manual coding.
- Lower development effort for common business applications.
- Easy prototyping and rapid testing of application ideas.
- Visual workflows make business processes easier to understand and modify.
- Built-in integrations and connectors can reduce integration effort.
- Applications can often be developed by both professional developers and trained business users.
- Cloud deployment can simplify application delivery and collaboration.

Challenges

- Vendor lock-in can occur when applications depend heavily on a particular platform.
- Advanced requirements may still require programming or platform-specific knowledge.
- Customization can be limited compared with a fully coded application.
- Security, access control, and governance must be managed carefully.
- Large or highly complex applications may require careful architecture and performance planning.
- Licensing and subscription costs can become significant depending on the platform and usage.
- Developers need to understand the platform's limitations and deployment model.

4. COMPARISON OF EXISTING LOW-CODE PLATFORMS

Platform	Main Use	Development Style	Integration	Automation	Suitable For
Zoho Creator	Business applications	Drag-and-drop + low-code scripting	APIs and connectors	Workflows and automation	SMEs and business teams
Microsoft Power Apps	Business apps	Visual low-code development	Microsoft and external connectors	Power Automate integration	Organizations using Microsoft ecosystem
OutSystems	Enterprise applications	Visual development + custom code	APIs and enterprise integrations	Workflow and automation features	Enterprise development
Mendix	Enterprise and business applications	Visual model-driven development	Connectors and APIs	Workflow automation	Enterprise teams
AppSheet	Data-driven mobile/web apps	No-code/low-code configuration	Google and external data sources	Bots and workflows	Small teams and business users

Corporate Asset Maintenance

CAM

Corporate Asset Maintenance

Department

Asset Details

Technician

AI fields

Maintenance Request

Maintenance Approvals

Repair Progress

Dashboard

Hotel Room Booking

Hotel Room Booking Report

Hotel Room Booking

ID_Proof_Number

Special_requests

Number of guest

checkin date

Preferred Checkin Time

Room type

meal plan

Amenities Required

Agree_to_hotel_policy

#####

dd-MMM-yyyy HH:mm:ss

dd-MMM-yyyy HH:mm:ss

-Select-

☐ room only

☐ breakfast included

☐ All meals

-Select-

☐ I agree

Submit

The survey shows that low-code application development is becoming an important approach for building business and data-driven applications. It reduces manual coding by providing visual development tools, reusable components, workflows, and integrations. Low-code platforms can improve development speed and accessibility, while traditional development continues to provide greater direct control for highly customized and complex software. Therefore, the choice of development approach depends on application requirements, customization needs, technical skills, security requirements, integration needs, and project scale.

